Tema 3.A (primera parte): Análisis de Software y modelado estructural

Este tema introduce el análisis de software utilizando el paradigma de la Programación Orientada a Objetos. Se presenta UML como herramientas clave para el modelado y se aborda en detalle el diagrama estructural por excelencia: el Diagrama de Clases.

Índice

1

Análisis de Software

- Análisis y Modelado Conceptual
- Paradigmas de Desarrollo
- Programación Orientada a Objetos
- Modelado en POO mediante UML

2

Modelado Estructural

- Diagramas de Clases
  - Introducción y fundamentos
  - Relaciones Estructurales
    - Asociación
    - Agregación
    - Composición
  - Herencia
- Diagramas de Objetos

Índice

1

Análisis de Software

- Análisis y Modelado Conceptual
- Paradigmas de Desarrollo
- Programación Orientada a Objetos
- Modelado en POO mediante UML

2

Modelado Estructural

- Diagramas de Clases
  - Introducción y fundamentos
  - Relaciones Estructurales
    - Asociación
    - Agregación
    - Composición
  - Herencia
- Diagramas de Objetos

Actividades del proceso software

Análisis de Software

El análisis de software consiste en el **estudio exhaustivo y sistemático** de un producto software a desarrollar, antes de su implementación. Se centra en **comprender profundamente el problema del mundo real que se debe resolver**, sin considerar aún los detalles técnicos de cómo se construirá la solución.

Ámbito del problema

El análisis se concentra exclusivamente en el contexto del mundo real donde existe la necesidad que motivó el desarrollo del software. Esto incluye comprender a fondo los procesos de negocio, las reglas del dominio, los actores involucrados y las restricciones existentes.

Construcción del Modelo

La construcción del modelo implica la creación de representaciones abstractas del problema real mediante el uso de técnicas específicas de modelado. Estas representaciones ayudan a visualizar la estructura y el comportamiento sistema, suponiendo la base para un diseño de software robusto y alineado con las necesidades del negocio.

Análisis = modelado conceptual

> **Independencia Técnica:** Enfoque exclusivo en el "qué" del problema, sin considerar el "cómo" de la implementación

¿Por qué es importante la independencia técnica?

- Permite centrarse en comprender el problema sin distracciones técnicas
- Facilita la comunicación con usuarios y stakeholders no técnicos
- Hace el modelo más duradero y adaptable a cambios tecnológicos
- Reduce el riesgo de soluciones prematuras e inadecuadas

Separación de Conceptos

El análisis se mantiene libre de decisiones de diseño e implementación

Perspectivas del modelado

Propiedades Estáticas

Modelado de la **estructura** del sistema: entidades, atributos, relaciones y restricciones que definen la arquitectura de datos

Propiedades Dinámicas

Modelado del **comportamiento**: procesos, flujos de datos, eventos y transformaciones que ocurren en el sistema

Principales paradigmas de desarrollo

Estructurado/Procedural

- **Separación** entre datos y procesos
- Enfoque en funciones y transformaciones
- Técnicas: ER, DFD, etc.
- Adecuado para: Sistemas de alto rendimiento, sistemas embebidos (hardware), SO, etc.
- Lenguajes representativos: C, Pascal, Cobol, Ada, Go, etc.

Orientado a Objetos

- **Integración** de datos y comportamiento
- Enfoque en objetos y sus interacciones
- Técnicas: diagramas UML (clases, secuencia, casos de uso, etc.)
- Adecuado para sistemas complejos y evolutivos como software de gestión empresarial, videojuegos y sistemas de gran escala.
- Lenguajes representativos: Java, C#, etc.

Paradigma Estructurado o Procedural

> Descomposición del sistema en funciones y procedimientos. El software se organiza en torno a **acciones y procesos** que transforman datos.

Técnicas de modelado

Modelo Entidad-Relación (ER)

Técnica para modelar la **estructura** de datos mediante entidades, atributos y relaciones entre ellas.

Diagramas de Flujo de Datos (DFD)

Técnica para modelar el **comportamiendo** a través de diagramas que muestran cómo fluyen y se transforman los datos a través de procesos.

Paradigma Orientado a Objetos (POO)

En el paradigma orientado a objetos, el sistema se analiza desde la perspectiva de los elementos del mundo real que lo componen. Todo lo que nos rodea puede ser considerado un objeto. Un objeto es una entidad que combina datos (sus propiedades o atributos) y comportamiento (sus métodos o funcionalidades). Cada objeto es una instancia de una clase, que actúa como un plano o plantilla para crear objetos con características y acciones similares.

Objeto

Representa una entidad del mundo real, con estado (datos/atributos) y comportamiento (métodos).

Clase

Una plantilla o molde que define la estructura (atributos) y el comportamiento (métodos) común de un conjunto de objetos.

¿Por qué POO?

La Programación Orientada a Objetos ofrece ventajas significativas al acercar el diseño del software a la lógica del mundo real, mejorando la comprensibilidad y la forma en que abordamos los problemas.

Se acerca al mundo real

Permite modelar sistemas de manera intuitiva, reflejando entidades y sus interacciones en el mundo real.

Aumenta la comprensibilidad

El código modular y bien estructurado facilita la lectura, el entendimiento y el mantenimiento del sistema.

Afronta problemas eficazmente

Su metodología se alinea con la forma natural de descomponer y resolver problemas complejos.

POO: Propiedades Fundamentales

Reutilización

Capacidad de usar componentes de software existentes en nuevos proyectos, optimizando el desarrollo.

- **Reducción del tiempo**: El software reutilizado no hay que crearlo
- **Reducción del coste**: El coste de reutilización es menor que el de desarrollo
- **Aumento de la calidad**: El software reutilizado ya ha sido probado previamente habiéndose detectado sus errores o al menos la mayoría de ellos

Encapsulamiento

Ocultación del estado y los datos de un objeto a través del concepto de clase.

- **Parte pública**: No hay restricción sobre su visibilidad. Cualquier objeto de otra clase puede verlo. Por ejemplo, ciertos métodos de una clase.
- **Parte privada**: Sólo lo ve el propio objeto de la clase. Por ejemplo, y como normal general, los atributos de una clase.

Protege la integridad de los datos y facilita el mantenimiento del código, permitiendo modificar su funcionamiento interno sin afectar al resto del sistema

Modelado en POO mediante UML

Diagramas de Comportamiento (o dinámicos)

- ~~Diagrama de Casos de Uso~~ (tema 2)
- Diagramas de interacción
  - Diagrama de Secuencia
  - Diagrama de Comunicación
- Diagrama de Estados
- Diagrama de Actividades

Diagramas Estructurales (o estáticos)

- Diagrama de Clases
- Diagrama de Objetos
- ~~Diagrama de Componentes~~ (tema 4)
- ~~Diagrama de Despliegue~~ (tema 4)

Índice

1

Análisis de Software

- Análisis y Modelado Conceptual
- Paradigmas de Desarrollo
- Programación Orientada a Objetos
- Modelado en POO mediante UML

2

Modelado Estructural

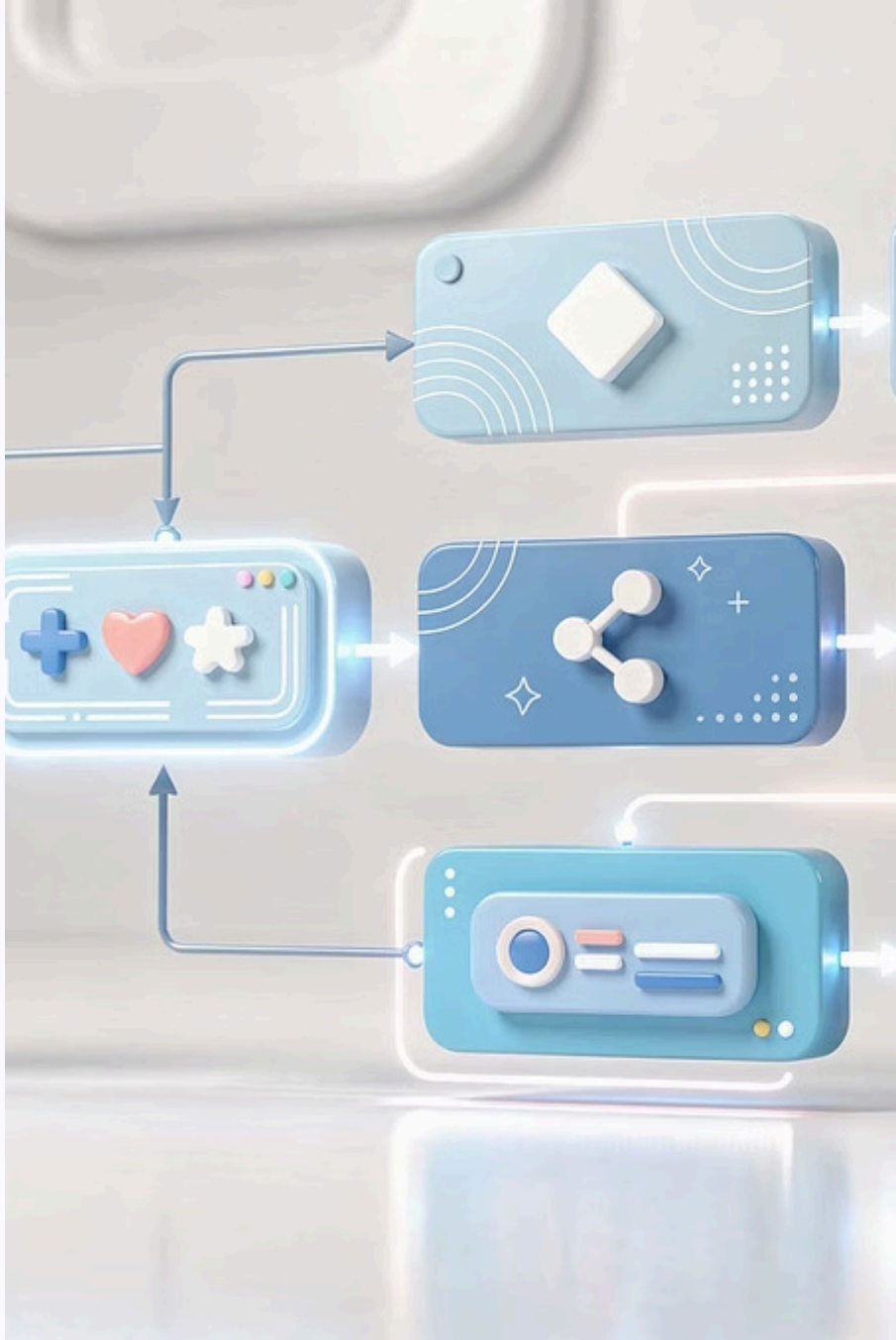
- Diagramas de Clases
  - Introducción y fundamentos
  - Relaciones Estructurales
    - Asociación
    - Agregación
    - Composición
  - Herencia
- Diagramas de Objetos

Diagramas de Clases

Un diagrama de clases representa la estructura estática de un sistema mediante clases y sus relaciones, tanto las estructurales como las de herencia.

Es el "plano arquitectónico" esencial que guía el diseño y desarrollo de software, permitiendo visualizar la organización completa del sistema antes de implementarlo.

En UML se representa como un grafo conexo que al menos tiene dos clases y una relación.



Importancia de los diagramas de clases

Los diagramas de clases son fundamentales para la visión estratégica de cualquier proyecto de software. Proporcionan una **representación clara y concisa de la estructura del sistema**, facilitando la comunicación efectiva entre equipos de desarrollo, analistas y stakeholders.

Permiten identificar relaciones complejas, anticipar problemas de diseño antes de la codificación y establecer una base sólida para la implementación. Su uso reduce la complejidad y asegura la coherencia a lo largo del ciclo de vida del software.

Documentación clara

Facilitan la comprensión profunda del sistema y su documentación técnica completa

Comunicación efectiva

Mejoran la comunicación entre desarrolladores, arquitectos y stakeholders del proyecto

Base de desarrollo

Sirven como fundamento para otros diagramas UML y para la implementación del código

¿Qué es una clase?

> Una clase es una abstracción de un conjunto de objetos con características comunes.

Una clase sirve como el "plano" conceptual para un grupo de objetos con propiedades y comportamientos idénticos. Es la piedra angular de la POO, permitiendo modelar elementos del mundo real o conceptos abstractos en el sistema.

Abstracción Fundamental

Define un conjunto de objetos que comparten características y funcionalidades.

Identidad Única

Cada clase posee un nombre distintivo dentro del diagrama para facilitar su referencia.

Creación de Objetos

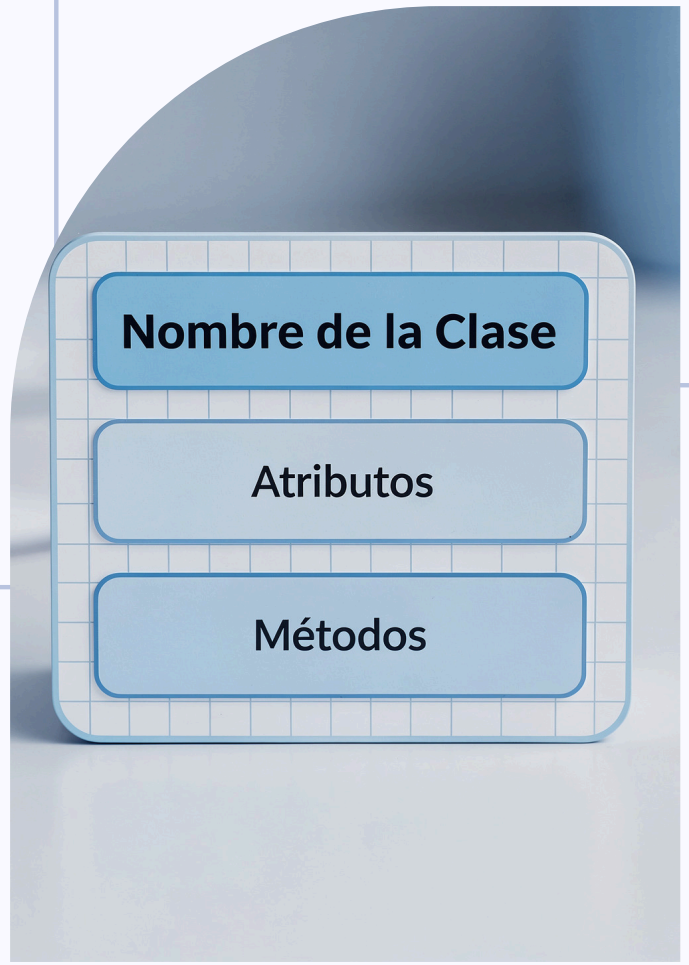
Los objetos son instancias concretas de una clase, cobrando vida a partir de su definición.

En UML una clase se define como un rectángulo con tres compartimentos

Anatomía de una clase en UML

Nombre de la Clase

Identificador único en la fila superior.



Atributos

Propiedades y datos de la clase, que definen a los objetos que representa la clase.

Métodos

Comportamientos y operaciones de la clase, que pueden ser solicitadas por otros objetos del sistema.

¿Cómo identificar clases?

La identificación de clases es un paso crucial para modelar un sistema. Se derivan principalmente del análisis del **documento de Especificación de Requisitos** (ERS) y los **escenarios de Casos de Uso**, que permiten la comprensión del dominio del problema. Algunas puntos clave a los que prestar atención son:

Sustantivos

En general, los sustantivos clave en la descripción de requisitos son a menudo candidatos ideales para clases.

Usuarios y Roles

Identifica los usuarios y los roles que desempeñan (p.e., gerente, ingeniero).

Estructuras y Contexto

Identifica unidades organizacionales (p.e., departamento, equipo) relevantes para el dominio.

Lugares

Identifica lugares (p.e., oficina, almacén) relevantes para el dominio.

Entidades y Roles

Identifica las entidades externas (p.e, sistemas, dispositivos) que interactúan con el sistema.

¿Cómo identificar métodos de una clase?

Los métodos son la funcionalidad que proporcionan los objetos de una clase, permitiendo la interacción y manipulación de sus datos. Definen el comportamiento y las acciones que los objetos pueden realizar dentro del sistema. Como representan acciones se especifican comúnmente mediante un **verbo** que describe el comportamiento que ejecutan (ej., `guardar()`, `calcularTotal()`).

Tipos Comunes de Operaciones

Manipulación de Datos

Operaciones para añadir, eliminar, reformatear o seleccionar información de los objetos.

Realización de Cálculos

Operaciones que procesan los datos para generar resultados específicos (ej., sumar, promediar).

Consulta de Estado

Métodos que preguntan sobre el estado actual de un objeto, sin modificarlo (ej., `estaVacio()`).

Monitoreo de Eventos

Operaciones que esperan y reaccionan ante la ocurrencia de un evento determinado en el objeto o sistema.

Métodos Esenciales: Constructor, destructor, getters y setters

Toda clase suele incluir un constructor (para crear objetos) y un destructor (para liberar recursos). Además, se usan métodos de consulta (getters) y modificación (setters) para acceder y modificar sus atributos.

Elementos de un Diagrama de Clases

Clases

Abstracciones de entidades del dominio o del sistema. Se representan como rectángulos divididos en tres compartimentos: nombre, atributos y operaciones.

Atributos

Propiedades estructurales que describen el estado o las características de los objetos de la clase.

Operaciones

Servicios que la clase ofrece y puede ejecutar. Definen la lógica y la interfaz de comunicación del objeto.

Relaciones

Líneas que conectan clases para representar vínculos semánticos (asociación), estructurales (agregación/composición) o de jerarquía (herencia).

Multiplicidad/Cardinalidad

Indicadores numéricos en los extremos de las relaciones (ej. 1, 0..*) que definen cuántas instancias participan en el vínculo.

Visibilidad

Marcas (+, -, #) que preceden a atributos y operaciones para definir su nivel de acceso desde otras clases (público, privado, protegido).

Alcance

Distinción entre miembros de instancia (propios de cada objeto) y miembros de clase (estáticos), que se representan subrayados.

Abstracción

Clases o métodos que actúan como plantilla base sin implementación completa (no instanciables), representados con su nombre en cursiva.

Visibilidad: Público, privado y protegido

Los atributos y métodos de una clase pueden tener distintos niveles de visibilidad.

Público (+)

Accesible desde cualquier clase del sistema sin restricciones

Por lo general los métodos son públicos, para que puedan ser llamados por otros objetos

Privado (-)

Accesible solo dentro de la propia clase, máxima encapsulación

Por lo general los atributos son privados, para mantener el principio de encapsulación y proteger su integridad

Protegido (#)

Accesible en la clase y sus subclases heredadas

Útiles cuando se trabaja con herencia y se quiere compartir funcionalidad solo con clases hijas

Alcance: de instancia o estático

Los atributos y métodos de una clase pueden tener distintos niveles de alcance.

Miembros de instancia o de clase

Son atributos o métodos únicos para cada objeto creado a partir de una clase.

Cada instancia mantiene su propia copia independiente de estos miembros, lo que significa que las modificaciones realizadas en los miembros de un objeto no afectan a los de otros.

Para poder acceder a ellos y utilizarlos, es indispensable crear un objeto o 'instanciar' la clase.

Por ejemplo, en un objeto 'Persona', el 'nombre' y la 'edad' serían miembros de instancia, ya que cada persona tiene sus propios valores individuales para estas propiedades.

Miembros estáticos

Son atributos o métodos que pertenecen a la clase en sí misma, no a objetos individuales o instancias de esa clase.

Esto significa que existe una única copia de un miembro estático que es compartida por todas las instancias. Estos miembros existen y están disponibles incluso antes de que se cree cualquier instancia de la clase.

Se pueden acceder directamente utilizando el nombre de la clase (p.e., `nombreClase.metodoEstatico()`).

En UML, los elementos estáticos se distinguen por estar subrayados.

Por ejemplo, en una clase 'Persona' puede haber un atributo ContadorDePersonas, que es un dato compartido por todas que nos dice cuántas existen.

Especificación de información en UML

La especificación de los detalles de una clase en UML en la **fase de análisis**, especialmente sus atributos y métodos, sigue ciertas convenciones para mantener la claridad y la concisión.

Nombre de atributos y de métodos

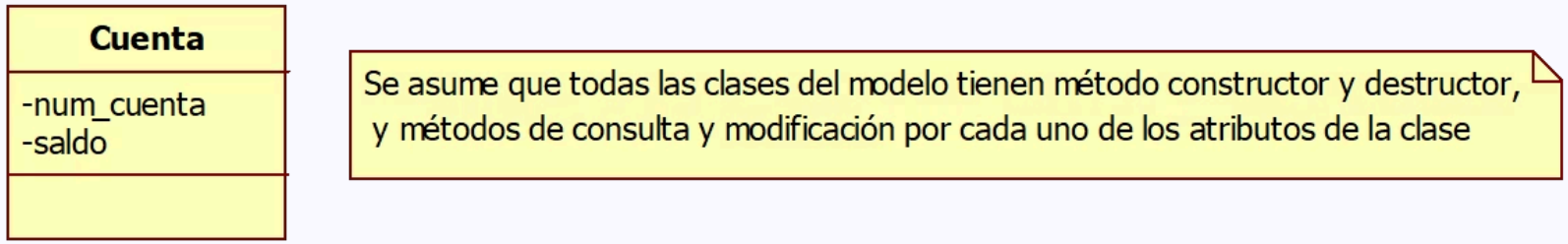
La especificación del nombre de todos los atributos y métodos es obligatoria pero...

Elementos no especificados

... no se especifican ni los **tipos de datos** de los atributos (salvo que sean un tipo enumerado) ni los **parámetros** de los métodos

Anotaciones Auxiliares

En diagramas muy extensos o complejos conviene aliviar el nivel de detalle usando anotaciones asociadas al diagrama de clases.



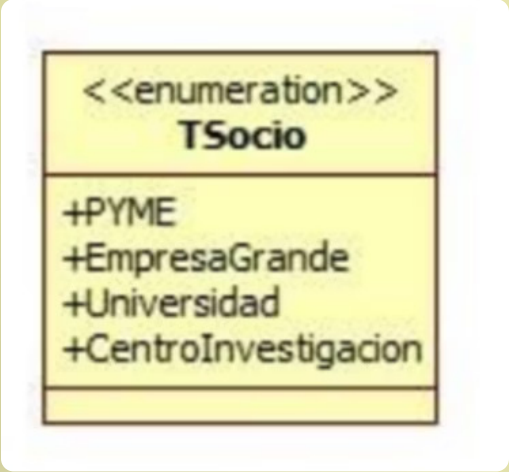
Tipo enumerado

Cuando el valor de un atributo pertenece a un conjunto cerrado de valores literales preestablecidos conviene indicarlo mediante un tipo enumerado.

Los tipos enumerados son un tipo de datos especial con un conjunto finito de valores constantes predefinidos. Útiles para representar estados, categorías o opciones limitadas.

En UML los enumerados se definen mediante el estereotipo «enumeration».

Por ejemplo, en un consorcio de investigación se pueden indicar los tipos de socios que lo conforman (PYME, Universidad, etc.).



Relaciones estructurales: Asociación, Agregación referencial y Agregación inclusiva (o composición)

Asociación

Relación estructural general entre clases que colaboran. Representa vínculos básicos sin dependencia de ciclo de vida.

- Navegación bidireccional.
- Normalmente binarias.
- Debe etiquetarse con un nombre único.
- Debe definirse su multiplicidad/cardinalidad.
- Pueden existir:
  - Relaciones reflexivas
  - Clases de asociación

En UML, se representa con una línea continua.

Agregación

Un tipo especial de asociación que representa una relación "todo-parte", donde un objeto (el "todo") está compuesto por otros objetos (las "partes"). La navegación es unidireccional (desde el "todo" a la "parte"). Se subdivide en dos tipos:

- **Agregación Referencial o Relacional**
  - Relación "todo-parte" débil donde las partes pueden existir independientemente del todo y ser compartidas por otros "todos".
- **Agregación Inclusiva / Composición**
  - Relación "todo-parte" fuerte con ciclo de vida compartido. Si se destruye el todo, las partes también dejan de existir (borrado en cascada).

En UML, se representa con una línea continua acompañada de un rombo hueco (agregación referencial) o relleno (composición) situado junto al "todo".

Multiplicidad y cardinalidad

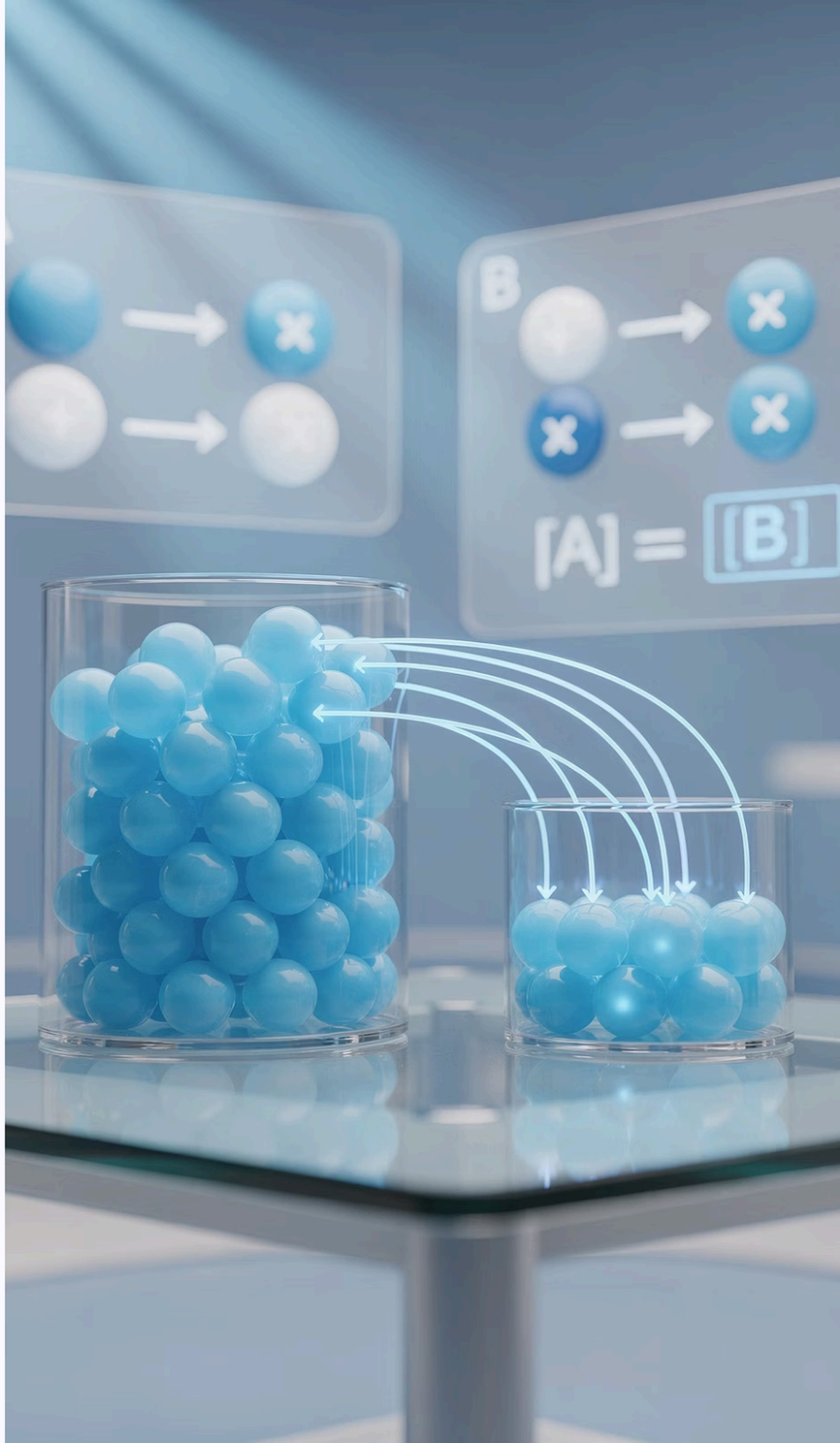
> La multiplicidad indica cuántas instancias de una clase pueden relacionarse con instancias de otra clase. Es fundamental para expresar reglas de negocio y restricciones del dominio.

Casos:

- **1:** Exactamente una instancia
- **0..1:** Cero o una instancia
- **1..*:** Una o muchas instancias
- **0..*:** Cero o muchas instancias
- **n..m:** Entre n y m instancias

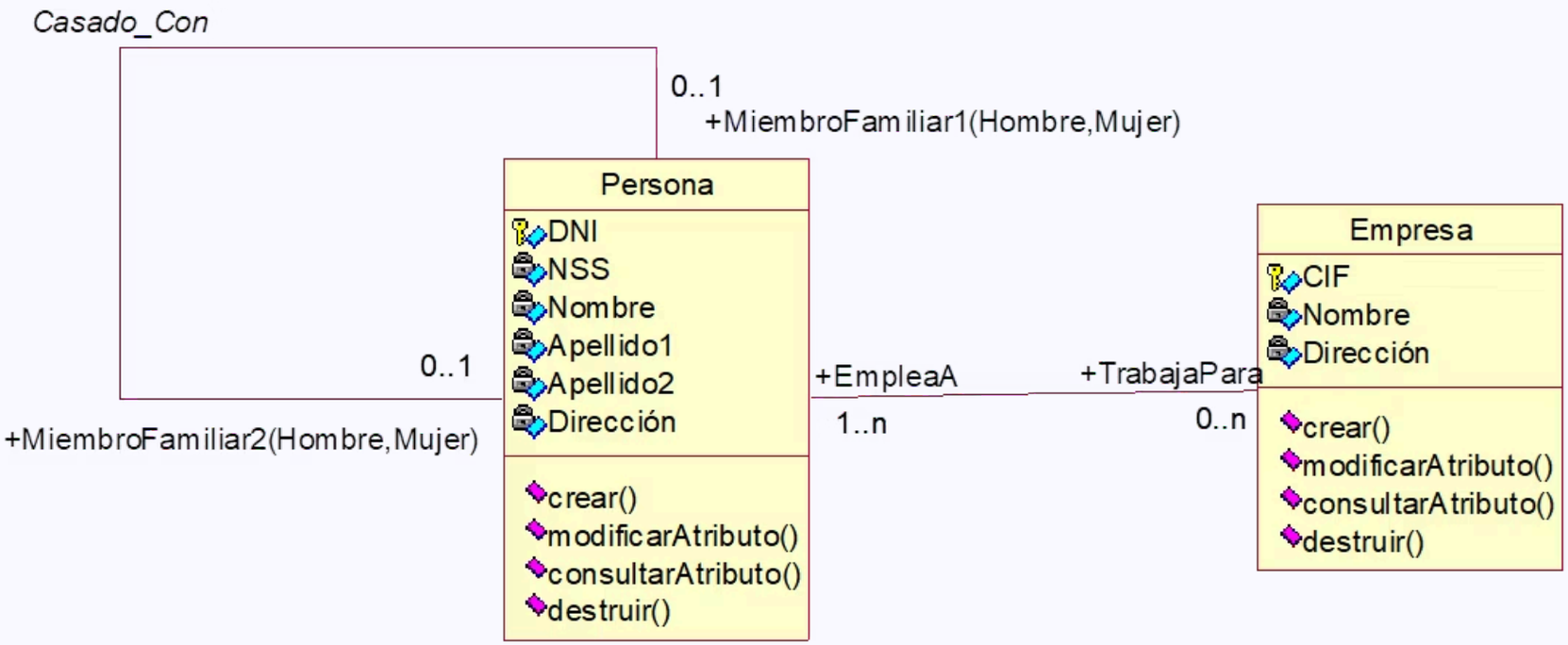
ⓘ Establecer las cardinalidades en un diagrama de clases es muy similar a establecerlas en un diagrama entidad-relación (bases de datos)

ⓘ Notese que la multiplicidad mínima y máxima puede establecer restricciones de existencia para los objetos de las clases asociadas.



Relaciones de asociación reflexivas

Una relación de asociación reflexiva es un tipo de asociación donde una clase se relaciona consigo misma, indicando que las instancias de esa clase pueden tener un vínculo con otras instancias de la misma clase, pero desempeñando diferentes roles. Este tipo de relación es común para modelar estructuras jerárquicas dentro de la misma entidad, como por ejemplo, un empleado que es gerente de otros empleados o una persona que se casa con otra.



En UML cada "extremo" de la relación se etiqueta con un rol y una multiplicidad para especificar cómo interactúan las instancias.

Relaciones de asociación: Clase Asociación

> Una Clase Asociación surge de una relación de asociación entre dos o más clases, cuando los enlaces entre estas necesitan tener propiedades, comportamientos o participar en otras relaciones como una entidad por derecho propio. Esencialmente, es una forma de modelar una relación como si fuera una clase, lo que permite añadir detalles que no caben en una asociación simple.

Guardar Información en Relaciones N:M

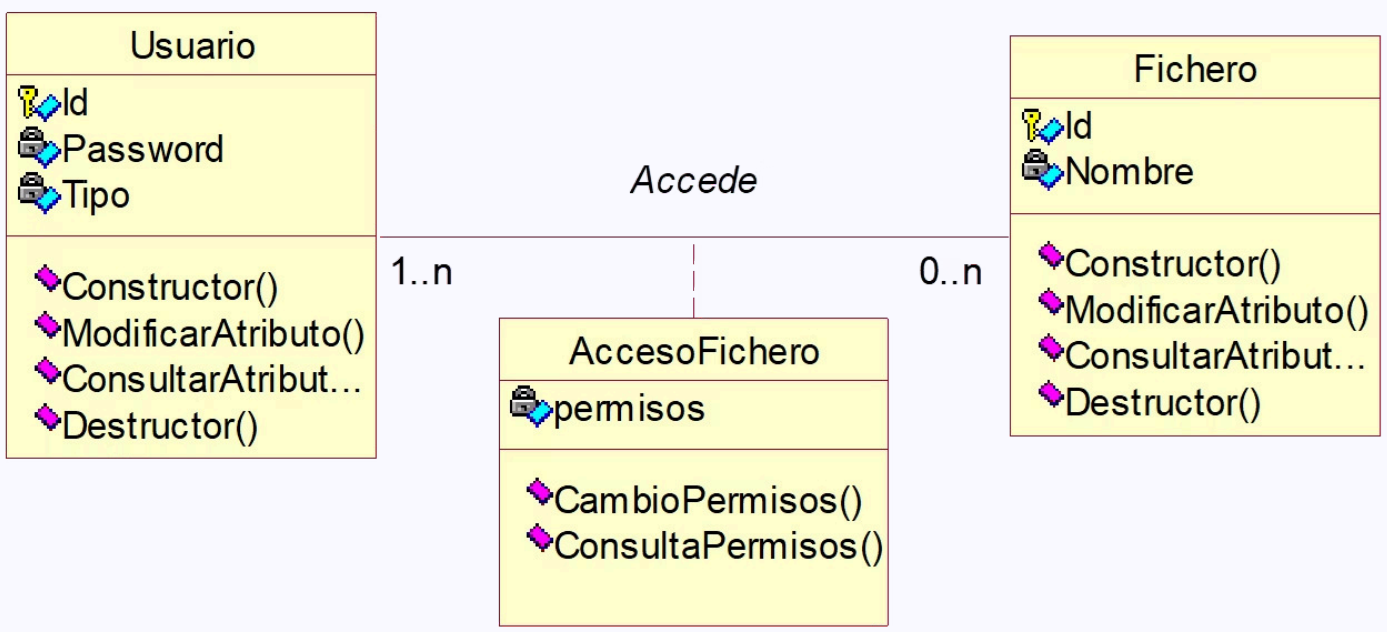
Permite almacenar atributos específicos del enlace que se forma en una relación muchos a muchos.

Provisión de operaciones

Se utiliza cuando los enlaces entre instancias de clases requieren comportamientos o funciones propias.

Participación en otras asociaciones

Cuando el enlace debe relacionarse con otras clases adicionales.



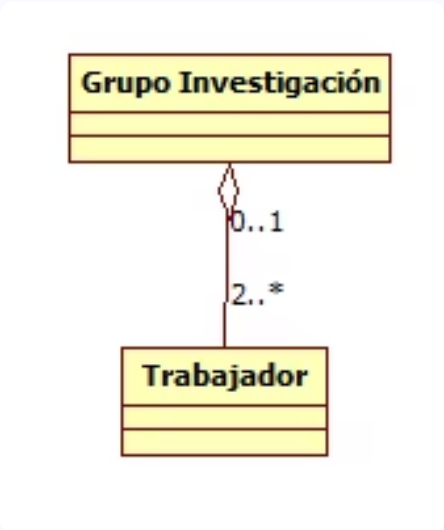
En UML se modela como una clase unida por una línea discontinua a la relación de asociación

Agregación referencial vs inclusiva

Agregación Referencial (Relacional)

Este tipo de agregación representa una relación "todo-parte" débil. Las partes pueden existir independientemente del todo y, en algunos casos, ser compartidas por múltiples "todos". No hay una dependencia estricta en el ciclo de vida; si el "todo" se destruye, las "partes" pueden seguir existiendo.

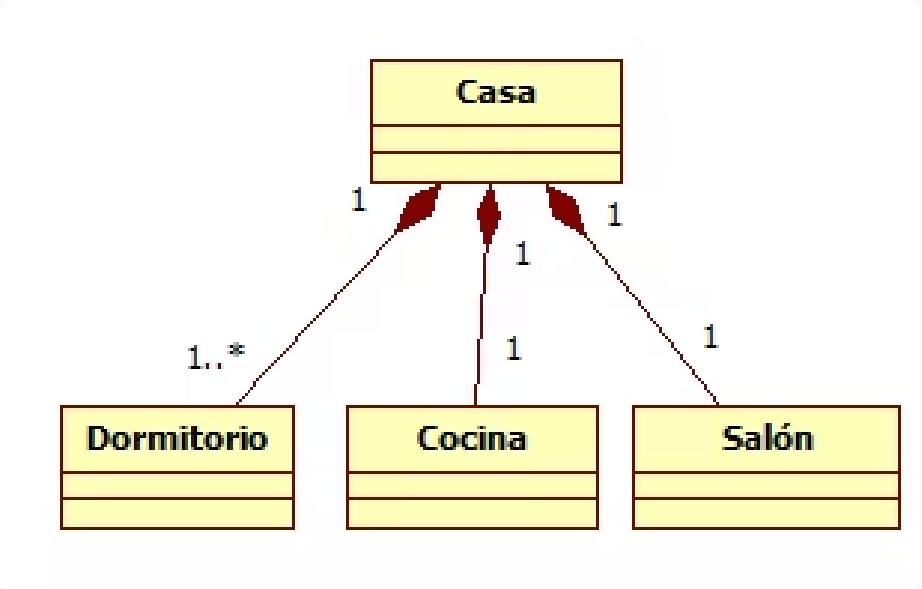
Ejemplo: Un grupo de investigación y sus miembros. Un trabajador puede existir antes de unirse a un grupo de investigación, puede cambiar de grupo o incluso existir si el grupo se disuelve. Puede ser "parte" de un grupo, pero no dependen de él para su existencia.



Agregación Inclusiva (Composición)

Esta es una forma fuerte de agregación, donde las "partes" son componentes esenciales del "todo" y no pueden existir de forma independiente. Si el "todo" se destruye, las "partes" asociadas a él también se destruyen (y se produciría un borrado en cascada). La composición implica una propiedad exclusiva del todo sobre sus partes.

Ejemplo: Una casa y sus habitaciones. Una habitación no puede existir como tal si la casa a la que pertenece es demolida. Las habitaciones son componentes integrales y dependen de la existencia de la casa.



Herencia

En la programación orientada a objetos, la herencia es un mecanismo fundamental que permite a una nueva clase, llamada clase hija (o clase derivada o subclase), adquirir las propiedades (atributos), comportamientos (métodos) y relaciones de una clase existente, conocida como clase padre (o clase base o superclase).

Semántica

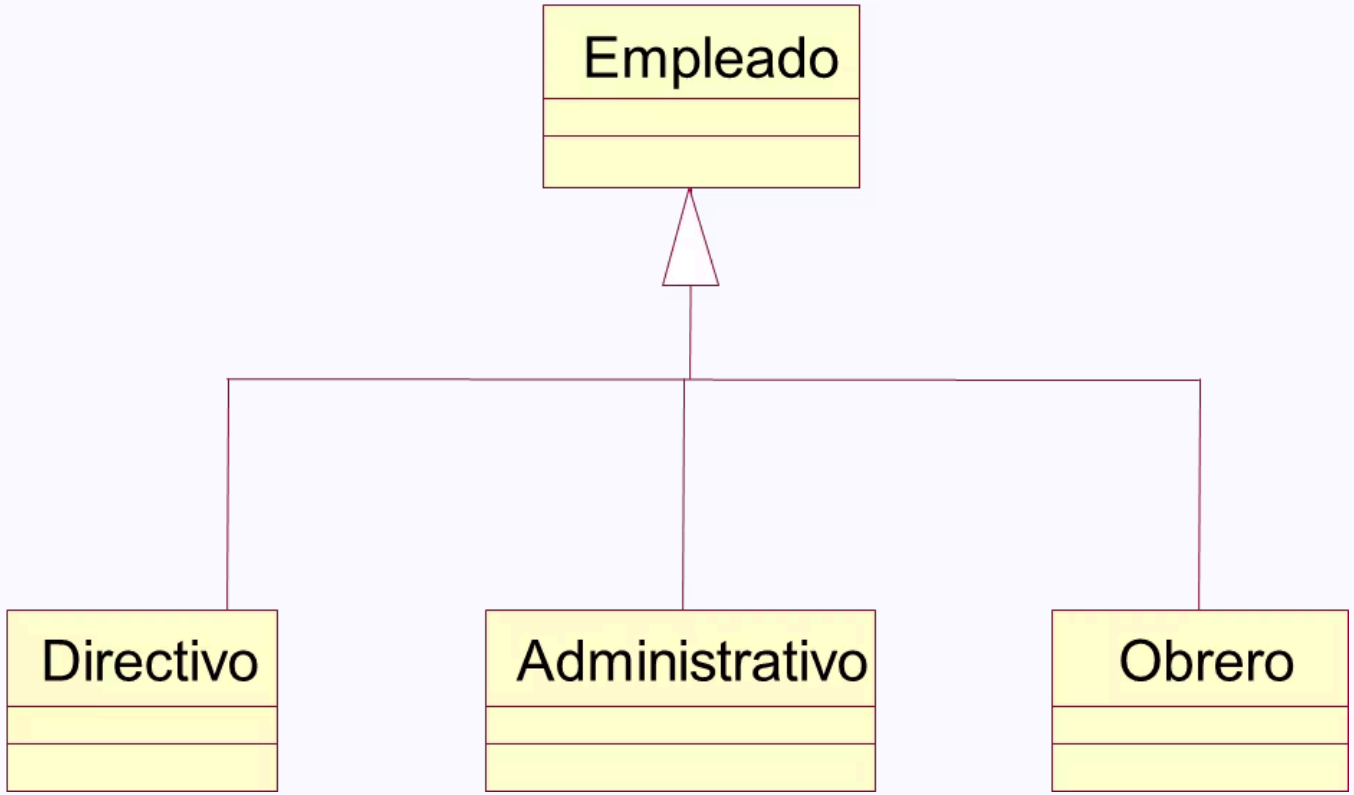
La herencia define una relación jerárquica 'es-un' (p.e., un Perro es un Animal) que surge para optimizar el diseño: ya sea por **Especialización** (añadiendo detalles a una base) o por **Generalización** (extrayendo lo común a un padre).

Elementos heredados

Las clases hijas adquieren automáticamente de sus clases padre:

- Atributos (propiedades)
- Operaciones (métodos/comportamientos)
- Relaciones con otras clases

Herencia: Ejemplo



En UML, la herencia se representa con una línea sólida y un triángulo hueco que apunta desde la clase hija hacia la clase padre.

Restricciones de Herencia

UML establece restricciones para modelar las relaciones de herencia con mayor precisión, definiendo cómo las subclases se relacionan con sus superclases.

Disjunta vs Solapada

Disjunta

Un objeto **NO** puede ser simultáneamente instancia de dos o más de las clases hijas que heredan de la misma superclase. Es una elección mutua exclusiva.

Solapada (No Disjunta)

Un objeto **SÍ** puede ser simultáneamente instancia de dos o más de las clases hijas. Las categorías de las subclases pueden superponerse.

Total vs Parcial

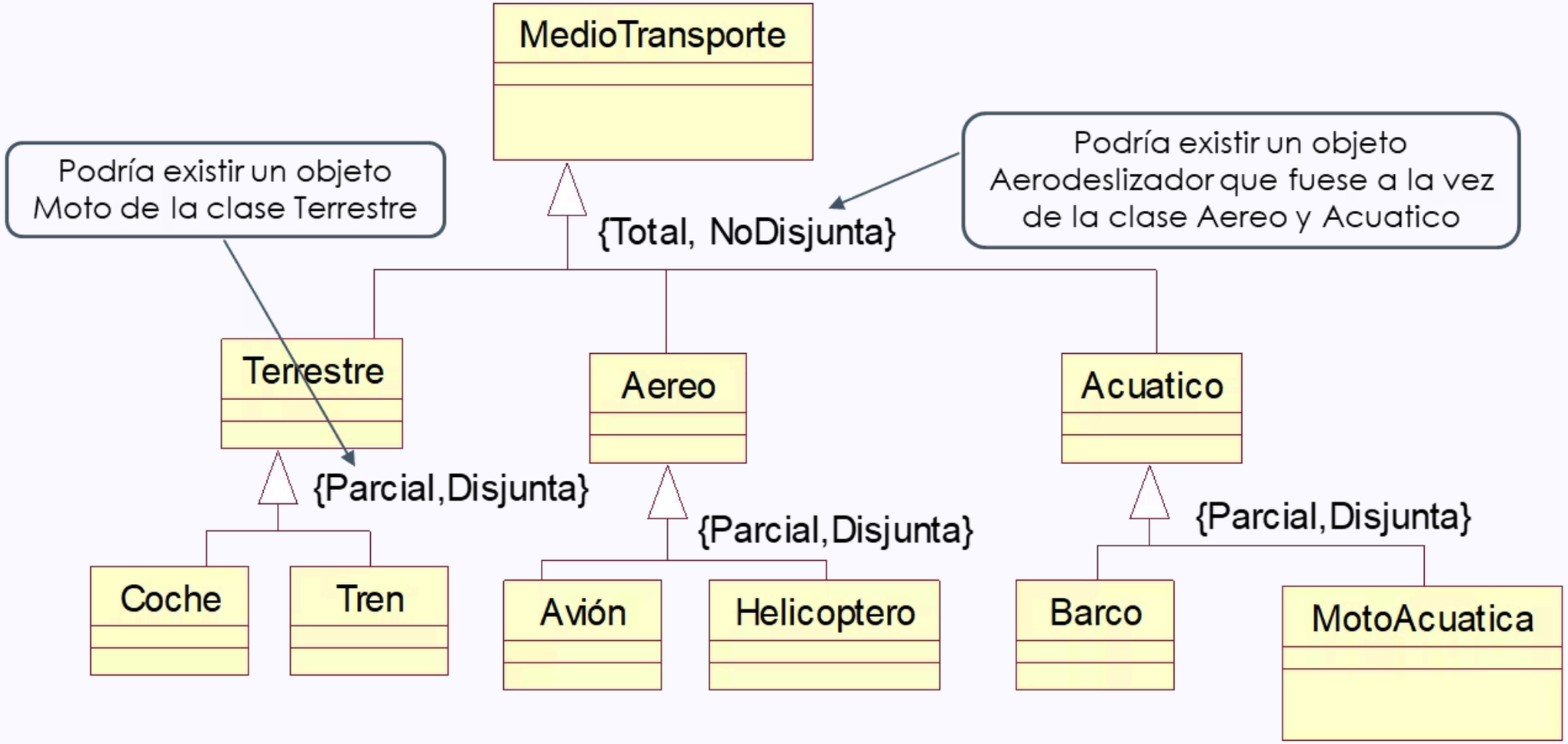
Total

La clase padre es **abstracta** y NO puede ser instanciada directamente. Todo objeto debe ser instancia de alguna de sus subclases.

Parcial

La clase padre es **concreta** y SÍ puede ser instanciada. Los objetos pueden ser directamente de la clase padre o de alguna de sus subclases.

Restricciones de Herencia: Ejemplo



Diagramas de Clases: Normas de creación

1

Nomenclatura y Formato

Clases: Sustantivos en singular (ej. `Pedido`).
Atributos: Sustantivos descriptivos (ej. `fecha`).
Métodos: Verbos o frases verbales (ej. `calcularTotal()`).
Compartimentos: La caja de clase tiene tres secciones: Nombre, Atributos y Métodos.

2

Encapsulamiento y Visibilidad

Atributos: Por defecto, Privados (`-`) o Protegidos (`#`) para encapsulamiento.
Métodos: Los servicios públicos de la clase deben ser Públicos (`+`).
Getters/Setters: Se indica su existencia mediante una nota y no se listan para evitar ruido visual.

3

Atributos

Tipos Primitivos: En análisis solo tipos primitivos (entero, cadena, fecha...) o enumerados.

La colección es la relación: No incluir listas de objetos como atributos (ej. `ListaPedidos`). Las colecciones se modelan con relaciones y multiplicidad múltiple en el extremo destino.

4

Relaciones vs. Atributos

Existen relaciones, no "claves foráneas": Las relaciones entre clases se representan con líneas de conexión, no con atributos de clave foránea (ej. ej. en la clase `Pedido` NO se pone el atributo `id_cliente`, si no una relación entre `Pedido` y `Cliente`)

5

Tipos de Relaciones

Distinguir relaciones bien:

Asociación: Relación genérica "conoce a".
Agregación: "Todo-Parte" débil (parte independiente).
Composición: "Todo-Parte" fuerte (parte muere con el todo).
Herencia: "es un tipo de" (subclase especializa superclase).

6

Multiplicidad

Definición: Cada extremo de una asociación debe definir su multiplicidad (ej. `1`, `0..1`, `1..*`, `*`). Indica cuántas instancias de una clase pueden vincularse con una instancia de la otra.
Impacto en implementación: Definirá el uso de variables simples o colecciones.

Diagramas de clases: Resumen

Introducción

Una clase es la "plantilla" para crear objetos, definiendo atributos (propiedades) y métodos (comportamientos). Se ha abordado cómo identificar clases, métodos, y la importancia de la visibilidad (público, privado, protegido) y el alcance (instancia, estático) de los atributos y métodos de una clase.

Relaciones Estructurales

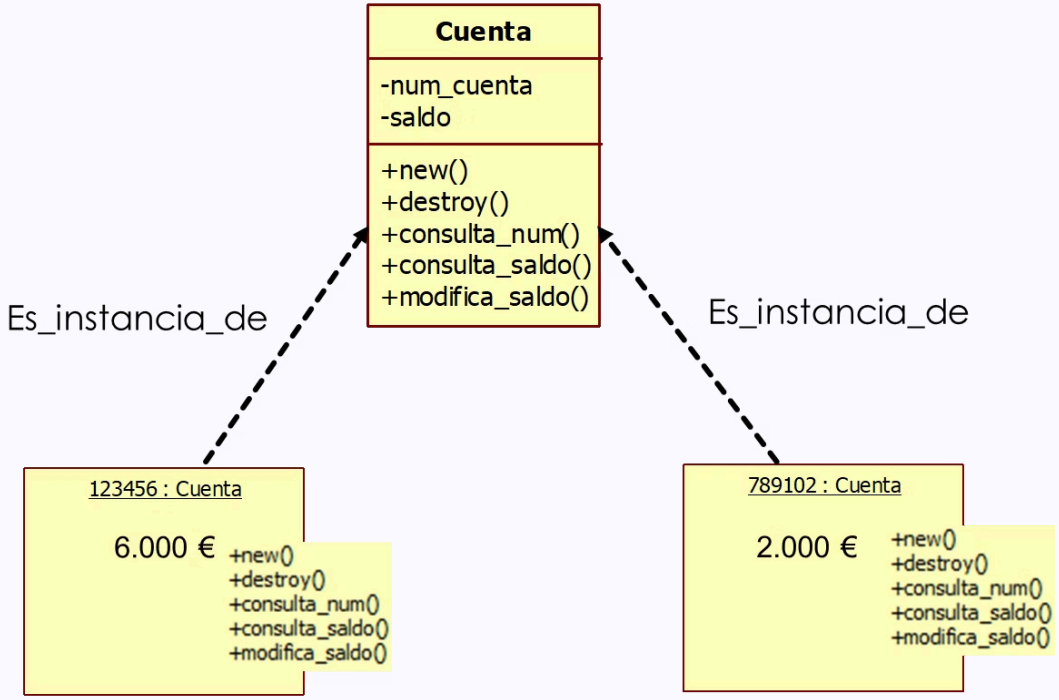
Las relaciones son vitales para mostrar cómo interactúan las clases. Exploramos la Asociación, la Agregación referencial, y la Agregación inclusiva o Composición. En todas ellas, resulta clave definir la multiplicidad (cardinalidad) que define cuántas instancias se pueden relacionar entre sí.

Relación de Herencia

La herencia modela la relación "es-un", permitiendo a las clases hijas adquirir características (atributos, métodos y relacionales) de sus clases padre. Exploramos también las restricciones como Disjunta vs Solapada y Total vs Parcial.

Diagrama de Objetos

Los objetos son instancias específicas de una clase en un momento determinado de la ejecución del sistema.

El diagrama de objetos representa una instantánea del sistema en un momento concreto, mostrando los objetos existentes, sus valores actuales y las relaciones entre ellos.

- En UML los objetos se representan con el nombre del objeto seguido de dos puntos y el nombre de la clase, todo subrayado (p.e., `miCoche:Coche`), incluyendo valores específicos de atributos, a diferencia de las clases que solo definen la estructura.